JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.function

Interface Function<T,R>

Type Parameters:

`T - the type of the input to the function`

`R - the type of the result of the function`

All Known Subinterfaces:

`UnaryOperator<T>`

Functional Interface:

`This is a functional interface and can therefore be used as the assignment target for a lambda expression or method reference.`

```
@FunctionalInterface
public interface Function<T,R>
```

Represents a function that accepts one argument and produces a result.

This is a functional interface whose functional method is `apply(Object)`.

Since:

`1.8`

Method Summary

Default Methods **Instance Methods** **Abstract Methods**

Modifier and Type	Method	Description
`default <V> Function<T,V>`		Returns a composed function that first

	andThen(**Function**<? super **R**,? extends V> after)	applies this function to its input, and then applies the after function to the result.
R	**apply**(**T** t)	Applies this function to the given argument.
default <V> **Function**<V,**R**>	**compose**(**Function**<? super V,? extends **T**> before)	Returns a composed function that first applies the before function to its input, and then applies this function to the result.
static <T> **Function**<T,T>	**identity**()	Returns a function that always returns its input argument.

Method Detail

apply

```
R apply(T t)
```

Applies this function to the given argument.

Parameters:

t - the function argument

Returns:

the function result

compose

```
default <V> Function<V,R> compose(Function<? super V,? extends T> before)
```

Returns a composed function that first applies the before function to its input, and then applies this function to the result. If evaluation of either function throws an exception, it is relayed to the caller of the composed function.

Type Parameters:

V - the type of input to the before function, and to the composed function

Parameters:

before - the function to apply before this function is applied

Returns:

a composed function that first applies the before function and then applies this function

Throws:

NullPointerException - if before is null

See Also:

andThen(Function)

andThen

```
default <V> Function<T,V> andThen(Function<? super R,? extends V> after)
```

Returns a composed function that first applies this function to its input, and then applies the `after` function to the result. If evaluation of either function throws an exception, it is relayed to the caller of the composed function.

Type Parameters:

V - the type of output of the after function, and of the composed function

Parameters:

after - the function to apply after this function is applied

Returns:

a composed function that first applies this function and then applies the after function

Throws:

NullPointerException - if after is null

See Also:

compose(Function)

identity

```
static <T> Function<T,T> identity()
```

Returns a function that always returns its input argument.

Type Parameters:

T - the type of the input and output objects to the function

Returns:

a function that always returns its input argument

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.